Technical Project Report: AI Crime Investigation & Suspect Identification System

Architecture: Decoupled (Django 5 REST API + React 19 SPA) | **Developer:** BCA / Junior Full-Stack & DevOps Portfolio

---

1. Project Overview

Project Name: AI-Based Crime Investigation & Suspect Identification System Using Deep Learning

Problem Solved: Replaces labor-intensive and error-prone human surveillance monitoring and manual forensic CCTV review with high-speed automated neural face identification.

Main Objective: Deliver an end-to-end operational platform capable of automated face detection, 512-dimensional feature embedding extraction, and suspect verification across live camera feeds and uploaded forensic video/image evidence.

Intended Users: Field Investigators (real-time stream scanning, evidence uploads, dossier downloads) and Administrators (investigator access control, suspect registration, model retraining).

2. Confirmed Technology Stack

Category	Technology	Verified Usage in Codebase
Backend	Python 3.11.9, Django 5.2.9	REST APIs, session authentication, ORM database operations
Frontend	React 19.2.0, Vite 7.3.7	Cyberpunk/HUD SPA, React Router 7, live polling, React Icons
AI / Deep Learning	TensorFlow 2.20, Keras-FaceNet 0.3.2	Inception-ResNet-v1 extracting 512-D L2-normalized embeddings
Vision & Alignment	MTCNN 1.0.0, OpenCV (cv2)	Face localization, eye-alignment affine rotation, Haar cascades
Database & Media	SQLite 3, WhiteNoise, Gunicorn	Persistent DB (db.sqlite3), static asset serving, WSGI server

3. Main Modules and Actual Purpose

1. Authentication & Role Control (api/auth_views.py): Dual-role session authentication separating Investigator badge logins from Admin staff accounts with PBKDF2 hashing.

2. Operations HUD Dashboard (pages/Dashboard.jsx): Central operational view displaying live camera status, system resource health, activity radar scores, and detection trends.

3. Live Camera Surveillance (pages/WebcamScan.jsx): On-demand MJPEG camera streaming (/api/video-feed/) with real-time bounding boxes, match confidence overlays, and audio alerts.

4. CCTV Evidence Processor (pages/VideoUpload.jsx): Uploads MP4/AVI videos or JPG/PNG photos, samples frames every 6 intervals, and matches candidates via temporal consensus.

5. Criminal Database Explorer (pages/CriminalDatabase.jsx): Searchable registry of registered criminals, mugshots, aliases, and mapped face_label identity keys.

6. Forensic Evidence Dossier (pages/EvidenceReport.jsx): Historical event log of positive detections with snapshots, exportable to formatted CSV or printable PDF views.

7. Administrative Command Center (pages/AdminDashboard.jsx): Full administrative CRUD for investigators, criminals, and crime records, with a one-click endpoint to retrain neural embeddings.

4. AI/ML Implementation Details

Face Detection & Alignment: Primary detection uses MTCNN to locate faces and extract eye keypoints. Roll angle is computed and corrected via affine rotation (cv2.warpAffine) if |theta| <= 30 deg. If MTCNN fails, secondary OpenCV Haar cascades are applied.

Quality Gate: Filters out frames failing brightness checks (40 <= mean <= 220) or sharpness thresholds (Laplacian variance >= 35.0).

Feature Extraction: Preprocessed 160x160 RGB crops pass into FaceNet (Inception-ResNet-v1), producing an L2-normalized 512-dimensional vector.

Cosine Matching: Distances are computed against the top-3 templates per registered identity in facenet_embeddings.pkl. Threshold is 0.62 (confidence >= 70%).

Dataset: 9 identity classes (person_001 to person_009) containing 221 total images (15 to 31 photos per subject).

Anti-Spoofing & Stability: Frame-difference motion scoring suppresses static photo attacks; a 7-frame rolling window requiring >= 4 hits eliminates transient false positives.

5. End-to-End Application Workflow

1. Login: User enters credentials -> Django verifies PBKDF2 hash -> sets session cookie -> React renders HUD.

2. Acquisition: User starts webcam or uploads video/image evidence.

3. Processing: Video frames are sampled -> faces detected by MTCNN -> eyes aligned -> 512-D embeddings generated.

4. Matching & Alerting: Embeddings compared via cosine distance -> if candidate meets confidence and temporal thresholds, backend logs RecognitionLog, generates AlertLog with captured snapshot, and emits live HUD notification.

5. Reporting: Officer inspects alert timeline on Evidence Report page and exports formal forensic dossier to CSV or printable PDF.

6. Feature Implementation Status

Feature	Verified Status	Code Evidence
Investigator & Admin Login	Implemented & Verified	api/auth_views.py:26
Role-Based Protected Routes	Implemented & Verified	ProtectedRoute.jsx, AdminProtectedRoute.jsx
Real-Time Webcam MJPEG Feed	Implemented & Verified	api/webcam_service.py:144
Motion Liveness & Temporal Filter	Implemented & Verified	api/webcam_service.py:168, 253
CCTV & Image Upload Analysis	Implemented & Verified	api/views.py:400
Suspect Database Browser	Implemented & Verified	pages/CriminalDatabase.jsx
Forensic Dossier Export (PDF/CSV)	Implemented & Verified	pages/EvidenceReport.jsx:80-265
Admin CRUD & Model Retraining	Implemented & Verified	api/auth_views.py:396
Pure WebRTC Media Streaming	Experimental / Incomplete	api/webrtc.py (SDP handshake only, no track)
Django HTML Templates	Deprecated / Legacy	Superseded by React 19 client SPA

7. Database and Backend Architecture

Database Models: Configured with SQLite in local development (crime_project/db.sqlite3). Models include Investigator (badge credentials, PBKDF2 hash), Criminal (profile, mugshot, face_label), CrimeRecord & Evidence (incident details, files), RecognitionLog (timestamped detections), and AlertLog (high-priority incident alerts with captured crime scene snapshots).

File Persistence: Criminal portraits saved to media/criminal_photos/, surveillance snapshots to media/alert_snapshots/, and uploaded videos to uploaded_videos/.

8. Portfolio Summary Information

Project Title: SENTINEL-AI: Automated Biometric Suspect Identification & Forensic Video Investigation System

Short Description: An enterprise-grade forensic investigation platform automating suspect identification across live surveillance streams and uploaded CCTV media using MTCNN and 512-dimensional FaceNet embeddings. Built on a decoupled architecture featuring a Django REST backend and a React 19 cybersecurity HUD interface.

Confirmed Technologies: Python, JavaScript, Django, React 19, Vite, TensorFlow, Keras-FaceNet, MTCNN, OpenCV, NumPy, SciPy, SQLite, WhiteNoise, Gunicorn.

9. Pending Improvements & Next Steps

1. Asynchronous Celery Task Queue: Transition large CCTV video processing from synchronous HTTP handlers to Celery + Redis workers to prevent timeouts on long files.

2. Vector Database Migration: Transition facenet_embeddings.pkl to pgvector or an indexed vector database for scalable similarity search across 100,000+ suspect records.

3. Hardware Acceleration: Implement ONNX Runtime or TensorRT export for FaceNet and MTCNN inference to achieve 60+ FPS on GPU servers.

4. Full WebRTC Binding: Bind aiortc video tracks to replace HTTP multipart MJPEG with low-latency peer-to-peer streaming.

5. Automated Test Suite: Introduce automated PyTest and Cypress test suites covering authentication, API contracts, and embedding distances.

10. Confirmation Required from Developer

1. Production DB Target: Confirm whether production will provision managed PostgreSQL or remain on SQLite.

2. WebRTC Scope: Confirm whether aiortc should be completed for ultra-low latency or archived in favor of MJPEG.

3. Hardware Target: Confirm whether demonstration benchmarks will run on CPU or a dedicated CUDA GPU.